Future Interns

API Security Risk Analysis Report

Name: SN Amogh

Internship Program: Future Interns Cybersecurity Internship

Task: API Security Risk Analysis Report

Tools Used: Postman, Browser Developer Tools, Kali Linux

Confidentiality Note:

This document contains the results of a security analysis conducted as part of the Future Interns Cybersecurity Internship Task 3. The information presented in this report is intended solely for educational and internship evaluation purposes.

The analysis was performed using publicly available demonstration APIs designed for testing and learning. No attempts were made to exploit vulnerabilities or access unauthorized systems. All testing activities were conducted in a controlled and ethical manner following responsible cybersecurity practices.

Unauthorized distribution or misuse of this report is not recommended.

Introduction

Application Programming Interfaces (APIs) play a crucial role in modern web applications by enabling communication between different software systems. APIs allow applications to retrieve data, perform operations, and interact with services efficiently. However, insecure APIs can expose sensitive data, allow unauthorized access, or be abused by attackers.

API security has become a critical concern in modern cybersecurity, especially with the growing number of applications relying on APIs for data exchange. Poorly secured APIs may lead to data breaches, unauthorized information disclosure, and service disruptions.

This report presents the findings from an API security risk analysis conducted on publicly available demonstration APIs. The analysis focuses on identifying common API security weaknesses such as lack of authentication, broken authorization controls, excessive data exposure, input validation issues, and absence of rate limiting mechanisms.

Objective

The primary objective of this task is to perform a security risk analysis on publicly accessible APIs and identify potential security weaknesses that could affect the confidentiality, integrity, or availability of systems.

The analysis aims to:

- Understand how APIs function and exchange data.
- Identify common API security risks based on industry standards.
- Evaluate API responses for potential data exposure.
- Test API endpoints for authorization and input validation issues.
- Document findings and recommend security improvements.

Tools Used

Postman

Postman is an API development and testing tool used to send HTTP requests and analyse responses from APIs. It allows testers to easily interact with API endpoints and inspect returned data.

Browser Developer Tools

Browser Developer Tools were used to inspect network requests and understand how APIs communicate with applications.

Kali Linux Environment

The testing environment was performed using Kali Linux, which provides a stable platform for cybersecurity research and analysis.

APIs Used in Testing

JSONPlaceholder API

Website:
https://jsonplaceholder.typicode.com

JSONPlaceholder is a fake REST API commonly used for testing and prototyping applications. It provides simulated user, post, and comment data that developers can use to test API integrations.

The API allows users to retrieve various types of data such as user profiles, posts, and comments through simple HTTP requests.

DummyJSON API

Website:
https://dummyjson.com

DummyJSON provides mock data related to users, products, and other entities for testing applications. It simulates realistic API responses and allows developers to experiment with various API endpoints.

The API returns structured JSON responses that include user details, product information, and search results.

ReqRes API

Website:
https://reqres.in

ReqRes is another publicly available API used for learning and testing RESTful API interactions. It provides simulated user data and supports various HTTP operations such as GET and POST requests.

Testing Methodology

The API security analysis was conducted using Postman to send HTTP requests to selected API endpoints. The responses returned by the APIs were examined to identify potential security weaknesses.

The following checks were performed during the testing process:

- Verification of authentication requirements for accessing API endpoints
- Authorization testing by modifying object identifiers in API requests
- Inspection of API responses for excessive data exposure
- Input validation testing using unusual query parameters
- Evaluation of API behaviour when multiple requests are sent rapidly

Screenshots were captured during testing to provide evidence of the observed responses.

Security Analysis and Evidence

This section presents the findings obtained during the API security testing process. Each API was tested using **Postman** to analyse responses, observe data exposure, and check for security controls such as authentication, authorization, input validation, and rate limiting.

1. JSONPlaceholder API Analysis

API Endpoint Tested

GET https://jsonplaceholder.typicode.com/users

The JSONPlaceholder API was tested to observe how user information is retrieved through a public API endpoint. The API returned a list of user records including details such as name, username, email address, phone number, website, and address information.

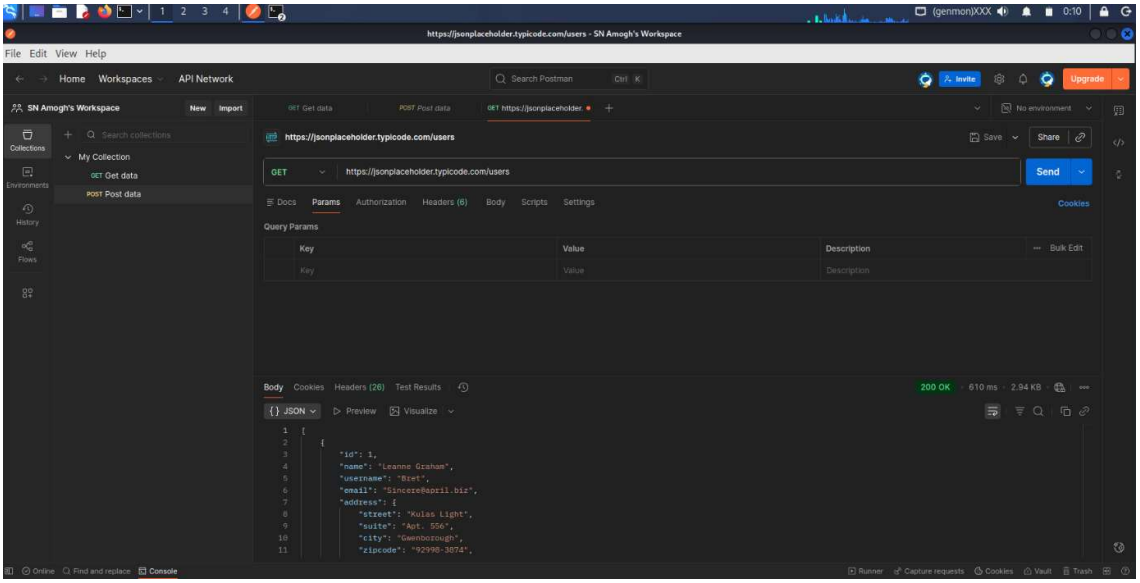

Figure 1: JSONPlaceholder API response displaying user data retrieved using Postman.

Observation:

The API endpoint allowed unrestricted access to user data without requiring any authentication mechanism. Any user can send a GET request to the endpoint and retrieve the full list of user records.

This behaviour demonstrates a **lack of authentication control**, which could become a security risk if sensitive data were exposed in a real-world application.

Authorization Testing

Additional testing was performed by modifying the user ID within the API endpoint.

Tested Endpoint:

GET https://jsonplaceholder.typicode.com/users/3

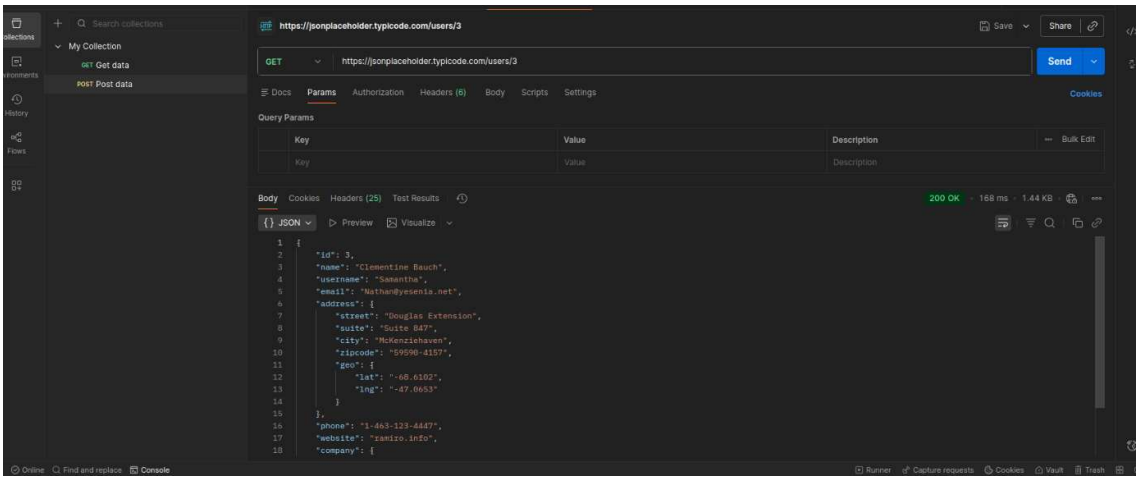

Figure 2: Accessing a specific user record by modifying the user ID in the API request.

Observation:

By changing the user ID value in the request URL, it was possible to access different user records without any authorization verification.

In a real-world system, this behaviour may indicate a **Broken Object Level Authorization (BOLA)** vulnerability. BOLA occurs

when APIs fail to properly validate whether a user has permission to access a particular resource.

Data Exposure Observation

The API response contains multiple fields such as address, phone number, website, and company information. While this API is a demo service, exposing unnecessary data fields could pose a risk in production environments.


```
{} JSON   Preview   Visualize
{
    "id": 3,
    "name": "Clementine Bauch",
    "username": "Samantha",
    "email": "Nathan@yesenia.net",
    "address": {
        "street": "Douglas Extension",
        "suite": "Suite 847",
        "city": "McKenziehaven",
        "zipcode": "59590-4157",
        "geo": {
            "lat": "-68.6102",
            "lng": "-47.0653"
        }
    },
    "phone": "1-463-123-4447",
    "website": "ramiro.info",
    "company": {
        "name": "Romaguera-Jacobson",
        "catchPhrase": "Face to face bifurcated interface",
        "bs": "e-enable strategic applications"
    }
}
```


Figure3: Example JSON response showing detailed user information including address and company data.

This behaviour demonstrates **Excessive Data Exposure**, which occurs when APIs return more information than required.

2. DummyJSON API Analysis

API Endpoint Tested

GET https://dummyjson.com/users

The DummyJSON API was analysed to observe how user profile information is returned through API responses.

The response contained multiple user attributes including: Name, Email, Phone number, Birth date, Username, Image URL, Blood group, Physical attributes

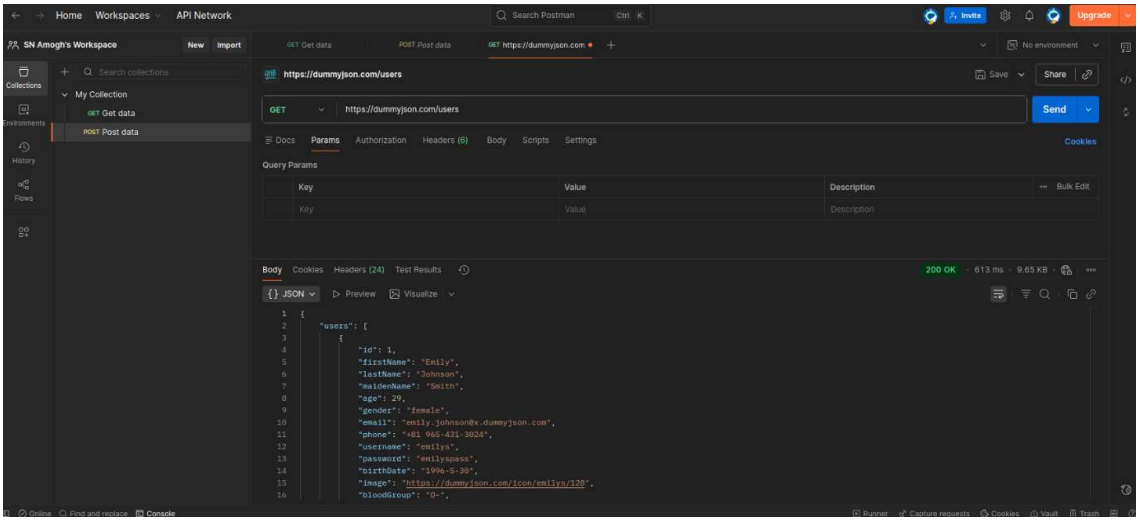

Figure 4: DummyJSON API response showing detailed user profile information.

Observation:

The API returned a large amount of personal data for each user. In real-world applications, exposing such detailed information without proper restrictions could increase the risk of information disclosure.

Authorization Testing

Testing was performed by modifying the user identifier in the API endpoint.

Tested Endpoints:

GET https://dummyjson.com/users/5

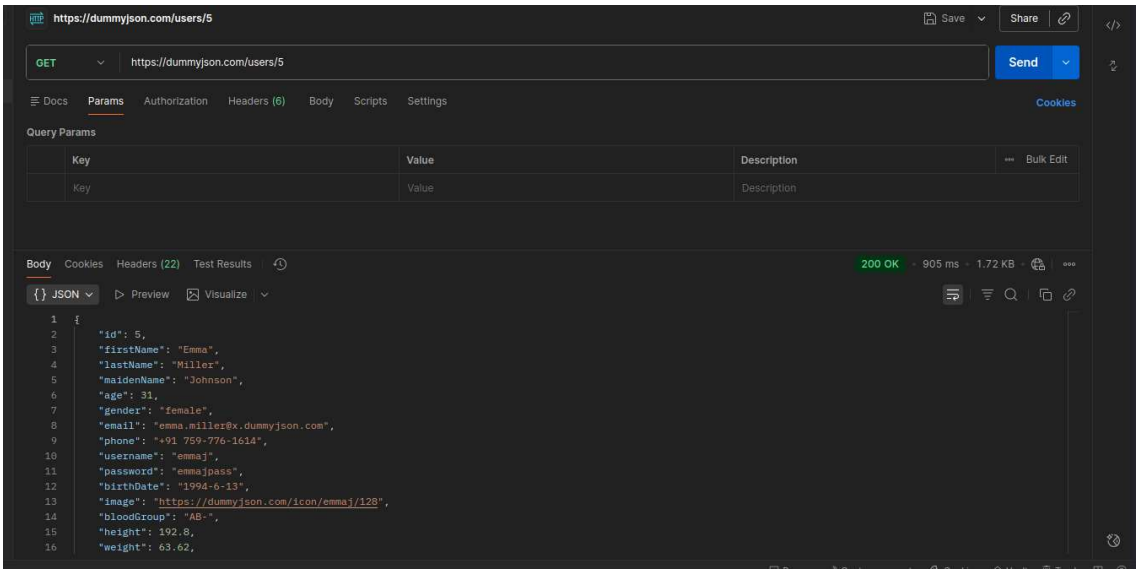

Figure 5: Accessing different user records by modifying the user ID parameter.

Observation:

The API returned the information of different users when the ID value was changed. This behaviour indicates that the API allows access to different user objects simply by modifying the identifier.

In real systems, this could represent a **Broken Object Level Authorization vulnerability**, where attackers may access other users' data by manipulating identifiers.

Input Validation Testing

Input validation testing was conducted by sending an unusual query parameter to the product search endpoint.

Tested Endpoint:

https://dummyjson.com/products/search?q=<script>

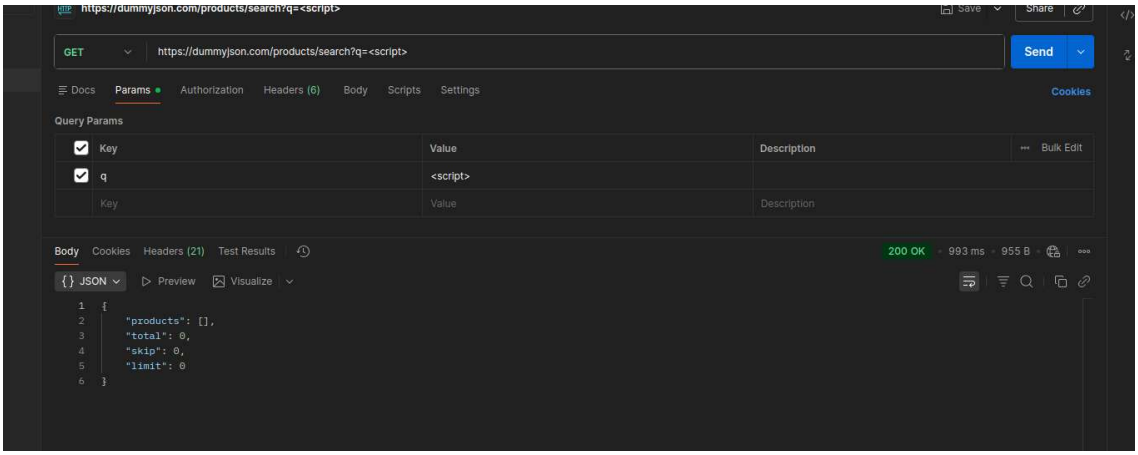

Figure 6: API response after submitting a modified query parameter containing special characters.

Observation:

The API accepted the query parameter and returned a valid response without filtering the input.

Although the API did not execute the script, accepting unusual characters without validation may indicate potential weaknesses in input handling mechanisms.

Improper input validation can potentially lead to vulnerabilities such as injection attacks in real-world applications.

3. ReqRes API Analysis

API Endpoint Tested

GET https://reqres.in/api/users?page=1

The ReqRes API was tested to analyse how the service responds to user data requests.

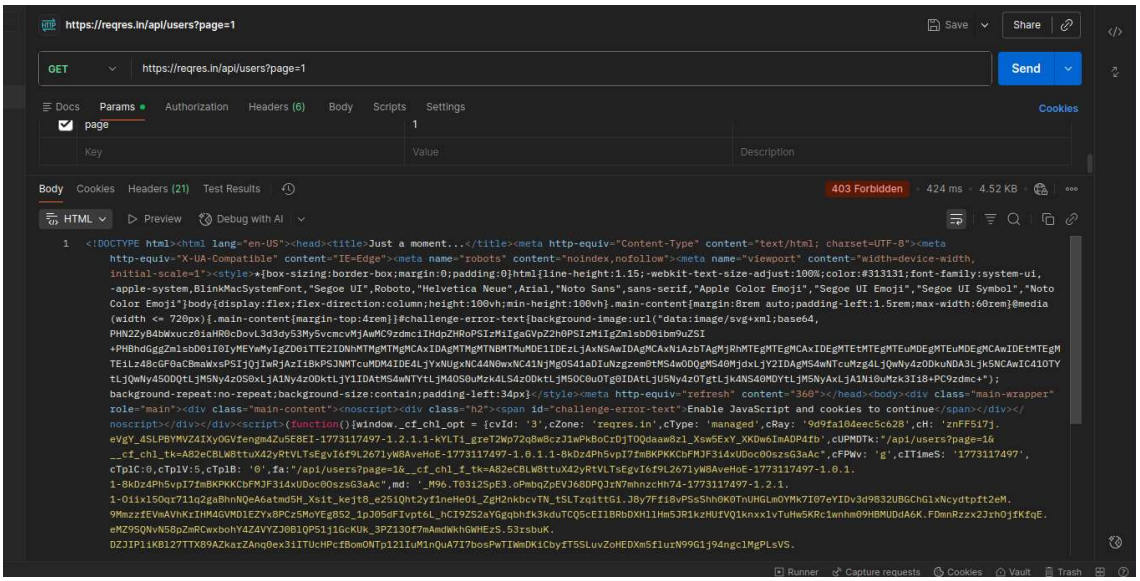

Figure 7: ReqRes API response displaying user information.

Observation:

The API returned a list of users including their ID, email address, first name, last name, and profile image.

The API endpoint does not require authentication, which means that any user can retrieve the data without verification.

Authorization Observation

Additional testing was conducted by requesting individual user records.

Example Endpoint:

GET https://reqres.in/api/users/5

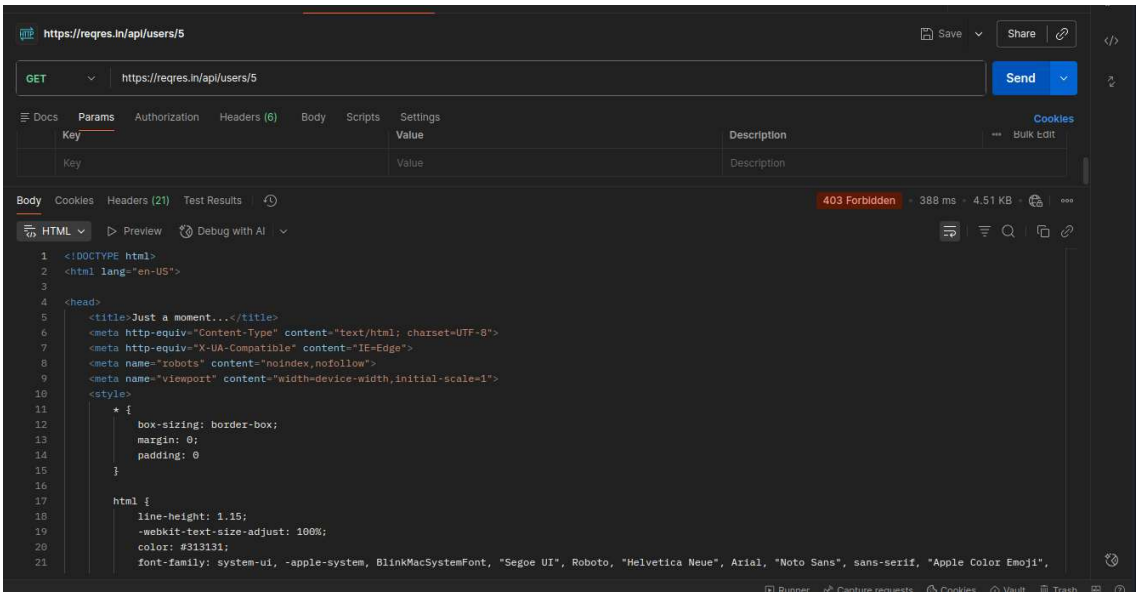

Figure 8: ReqRes API returning a 403 Forbidden response when attempting to access restricted resources.

Observation:

The server returned a **403 Forbidden response**, indicating that access to this resource was restricted.

This behaviour suggests that the API may implement access control mechanisms that prevent direct retrieval of certain resources.

Rate Limiting Test

Repeated requests were sent to the API endpoint to observe whether the server would limit the number of requests.

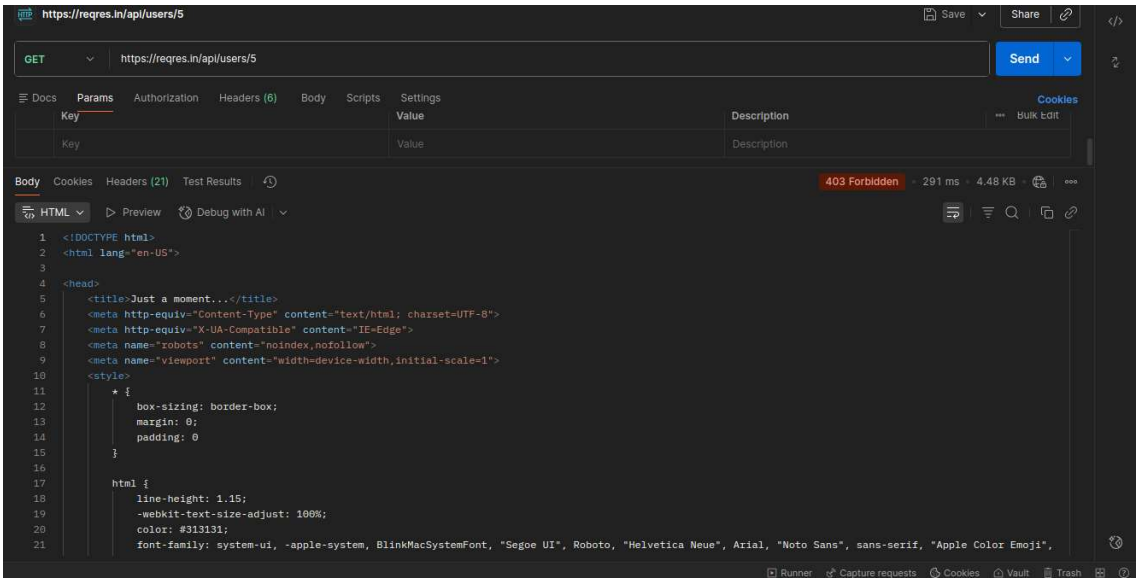

Figure 9: Multiple API requests sent without visible rate limiting restrictions.

Observation:

The API continued responding to requests without blocking repeated calls during the testing period. This behaviour indicates the possible absence of strict rate limiting controls.

In real-world scenarios, the absence of rate limiting could allow attackers to perform automated attacks or abuse the API.

Mapping Findings to OWASP API Security Top 10

The findings observed during the API security analysis can be mapped to several categories defined in the **OWASP API Security Top 10**, which is a widely recognized industry standard for identifying common API security risks.

OWASP provides guidelines to help organizations understand and mitigate vulnerabilities that commonly affect APIs.

The following table maps the observations from this analysis to relevant OWASP API security risks.

OWASP API Risk Mapping Table

Observed Behaviour	Description	OWASP Category
Accessing user records by modifying IDs	Changing object identifiers allowed retrieval of other users' data	API1: Broken Object Level Authorization
API endpoints accessible without authentication	Users could access API endpoints without providing credentials	API2: Broken Authentication
APIs returning large amounts of user data	Responses included unnecessary fields such as phone numbers and address details	API3: Excessive Data Exposure
Acceptance of unusual input parameters	API accepted inputs like <script> without filtering	API8: Injection / Improper Input Validation
Multiple requests without restrictions	API continued responding without blocking repeated requests	API4: Lack of Resources & Rate Limiting

Limitations

The analysis conducted in this report was limited to publicly available demonstration APIs designed for educational purposes. These APIs are intentionally configured to allow easy access to data and may not represent real-world security implementations.

Additionally, the testing was limited to read-only requests and did not involve advanced penetration testing techniques such as automated scanning, authentication bypass attempts, or exploitation of vulnerabilities.

Due to these limitations, the findings presented in this report should be considered as an educational demonstration of common API security risks rather than a comprehensive security assessment.

Conclusion

The API security risk analysis conducted in this task provided valuable insights into how APIs function and how security weaknesses can appear in API implementations.

During the analysis, several common API security concerns were observed, including lack of authentication, excessive data exposure, acceptance of unusual input parameters, and limited request control mechanisms.

Although the APIs used in this task were designed for demonstration purposes, the testing highlights the importance of implementing proper API security controls in production environments.

Organizations should adopt secure API development practices, including strong authentication mechanisms, proper authorization checks, minimal data exposure, input validation, and rate limiting controls. Regular API security assessments can help identify and mitigate potential risks before they impact users or systems.